Laporan Praktikum
Praktikum Pengujian Perangkat Lunak
Pertemuan 2
Life Cycle

Nama Mahasiswa: Januarsyah Akbar (24/535846/SV/24314)
Kelas B2
Dosen Pengampu: Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.

February 24, 2026

Contents

1 Tujuan Praktikum **2**

2 Langkah Praktikum **3**

2.1 Unit Test: WalletTest.java 3

2.1.1 BeforeAll 3

2.1.2 BeforeEach 3

2.1.3 AfterAll 4

2.1.4 AfterEach 4

2.1.5 Test 4

3 Kesimpulan **8**

Chapter 1

Tujuan Praktikum

1. Mahasiswa memahami life cycle dan anotasi yang ada pada JUnit
2. Mahasiswa mampu mengimplementasikan anotasi pada pengujian unit

Chapter 2

Langkah Praktikum

Resources

Download source code praktikum pada link berikut:
`https://github.com/januarsyah901/PPPL/tree/main/Pertemuan%202`

2.1 Unit Test: WalletTest.java

2.1.1 BeforeAll

Metode `setUp()` yang dianotasi dengan `@BeforeAll` dijalankan sekali sebelum semua test method dalam class dijalankan. Metode ini bersifat statis dan digunakan untuk inisialisasi resource yang didapatkan bersama oleh semua test.

```java
@BeforeAll
static void setUp() {
    staticSharedWallet = new Wallet();
    staticSharedWallet.setOwner("Shared Wallet for All Tests");
    staticSharedWallet.addMoney(500_000);
    staticSharedWallet.addCard("MANDIRI Debit");
    staticSharedWallet.addCard("BCA Visa");
}
```

Pada contoh ini, method `setUp()` membuat sebuah instance `Wallet` statis yang dibagikan untuk semua test. Wallet ini diinisialisasi dengan nama pemilik, saldo awal 500.000, dan dua kartu kredit.

2.1.2 BeforeEach

Metode `setUpEachTest()` yang dianotasi dengan `@BeforeEach` dijalankan sebelum setiap test method dijalankan. Metode ini digunakan untuk menginisialisasi resource yang diperlukan oleh setiap test secara individual.

```java
@BeforeEach
void setUpEachTest() {
    wallet = new Wallet();
    wallet.setOwner("Kadal Coklat");
    wallet.addMoney(100_000);
    wallet.addCard("BRI BritAma");
}
```

Untuk setiap test, dibuat instance Wallet baru dengan pemilik "Kadal Coklat", saldo awal 100.000, dan satu kartu debit. Ini memastikan setiap test memiliki state yang fresh dan tidak terpengaruh oleh test sebelumnya.

2.1.3 AfterAll

Metode `cleanUp()` yang dianotasi dengan `@AfterAll` dijalankan sekali setelah semua test method selesai dijalankan. Metode ini digunakan untuk membersihkan resource global.

```
@AfterAll
static void cleanUp() {
    staticSharedWallet = null;
}
```

Method ini membebaskan reference ke `staticSharedWallet` dengan mengatur nilainya menjadi null, memastikan resource global dibersihkan setelah semua test selesai.

2.1.4 AfterEach

Metode `tearDownEachTest()` yang dianotasi dengan `@AfterEach` dijalankan setelah setiap test method selesai dijalankan. Metode ini digunakan untuk membersihkan resource lokal.

```
@AfterEach
void tearDownEachTest() {
    wallet = null;
}
```

Setelah setiap test selesai, object `wallet` yang telah digunakan dibebaskan dengan mengatur nilainya menjadi null. Ini mencegah memory leak dan memastikan state bersih untuk test berikutnya.

2.1.5 Test

Bagian ini berisi lima test method yang masing-masing menguji fungsionalitas berbeda dari class Wallet.

Order Annotation

Untuk mengontrol urutan eksekusi test method, digunakan anotasi `@Order` bersama dengan `@TestMethodOrder`. Pada class level, ditambahkan:

```
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class WalletTest {
    ...
}
```

Kemudian, setiap test method dianotasi dengan `@Order(n)` untuk menentukan urutan eksekusinya:

```
@Test
@Order(1)
void testAddAndTakeMoney() { ... }

@Test
@Order(2)
void testAddAndTakeCard() { ... }
```

Anotasi `@TestMethodOrder(MethodOrderer.OrderAnnotation.class)` memberi tahu JUnit untuk menjalankan test method sesuai dengan urutan angka yang ditetapkan pada anotasi `@Order`. Nilai yang lebih kecil dijalankan terlebih dahulu. Ini berguna ketika test-test memiliki ketergantungan atau ketika perlu memastikan test dijalankan dalam urutan tertentu.

testAddAndTakeMoney()

Test method ini menguji fungsionalitas penambahan dan pengambilan uang dari wallet.

```
@Test
@Order(1)
void testAddAndTakeMoney() {
    assertEquals(100_000, wallet.getTotalMoney());
    wallet.addMoney(50_000);
    assertEquals(150_000, wallet.getTotalMoney());
    boolean success = wallet.takeMoney(70_000);
    assertTrue(success);
    assertEquals(80_000, wallet.getTotalMoney());
    success = wallet.takeMoney(100_000);
    assertFalse(success);
    assertEquals(80_000, wallet.getTotalMoney());
}
```

Test ini memverifikasi bahwa saldo awal wallet adalah 100.000. Kemudian, penambahan 50.000 menghasilkan saldo 150.000 sebagaimana diharapkan. Selanjutnya, pengambilan 70.000 berhasil dilakukan dan saldo menjadi 80.000. Terakhir, pengambilan 100.000 gagal dilakukan karena saldo tidak mencukupi, dan saldo tetap berada di 80.000.

testAddAndTakeCard()

Test method ini menguji fungsi penambahan dan pengambilan kartu dari wallet.

```
@Test
@Order(2)
void testAddAndTakeCard() {
    assertEquals(1, wallet.getCards().size());
    wallet.addCard("BNI Taplus");
    wallet.addCard("OVO");
    assertEquals(3, wallet.getCards().size());
    assertTrue(wallet.getCards().contains("BNI Taplus"));
    String taken = wallet.takeCard("OVO");
    assertEquals("OVO", taken);
    assertFalse(wallet.getCards().contains("OVO"));
    assertEquals(2, wallet.getCards().size());
}
```

Test ini memverifikasi bahwa wallet awalnya memiliki 1 kartu. Setelah penambahan 2 kartu baru (”BNI Taplus” dan ”OVO”), jumlah total kartu menjadi 3. Kemudian, test memastikan bahwa kartu yang baru ditambahkan berhasil ditemukan dalam wallet. Terakhir, pengambilan kartu ”OVO” berhasil dilakukan, menghapusnya dari wallet sehingga sisa kartu kembali menjadi 2.

testTakeNonExistentCard()

Test method ini menguji pengambilan kartu yang tidak ada di wallet.

```
@Test
@Order(3)
void testTakeNonExistentCard() {
    String result = wallet.takeCard("Gopay");
    assertNull(result);
    assertEquals(1, wallet.getCards().size());
}
```

Test ini memverifikasi bahwa ketika mencoba mengambil kartu yang tidak ada ("Gopay"), method mengembalikan null dan jumlah kartu tidak berubah.

testOwner()

Test method ini menguji fungsionalitas get dan set pemilik wallet.

```
@Test
@Order(4)
void testOwner() {
    assertEquals("Kadal Coklat", wallet.getOwner());
    wallet.setOwner("Janu Si Kadal");
    assertEquals("Janu Si Kadal", wallet.getOwner());
}
```

Test ini memverifikasi pemilik awal sesuai dengan yang ditetapkan di BeforeEach, dan pengubahan pemilik berhasil disimpan dan dapat diambil kembali.

testStaticWalletStillExists()

Test method ini menguji bahwa wallet statis yang dibuat di BeforeAll masih ada dan tetap mempertahankan nilainya.

```
@Test
@Order(5)
void testStaticWalletStillExists() {
    assertNotNull(staticSharedWallet);
    assertEquals("Shared Wallet for All Tests", staticSharedWallet.getOwner());
    assertEquals(500_000, staticSharedWallet.getTotalMoney());
    assertEquals(2, staticSharedWallet.getCards().size());
}
```

Test ini memverifikasi bahwa wallet statis yang dibagi tidak null dan tetap mempertahankan nilai yang ditetapkan di BeforeAll, menunjukkan bahwa resource global tidak terpengaruh oleh test-test individual.

Test Passed

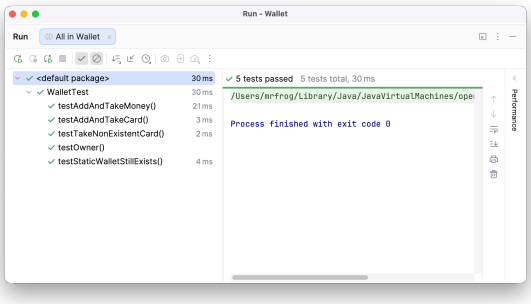

Figure 2.1: Hasil test yang berhasil

Chapter 3

Kesimpulan

Pada praktikum ini, saya belajar tentang anotasi yang ada pada JUnit, seperti BeforeAll, BeforeEach, AfterAll, AfterEach, dan Test. Anotasi BeforeAll digunakan untuk mengeksekusi kode sebelum semua pengujian dijalankan, sedangkan BeforeEach digunakan untuk mengeksekusi kode sebelum setiap pengujian dijalankan. Anotasi AfterAll digunakan untuk mengeksekusi kode setelah semua pengujian selesai dijalankan, sedangkan AfterEach digunakan untuk mengeksekusi kode setelah setiap pengujian selesai dijalankan.